SkillSphere

Campus Skill Exchange Marketplace

College of Engineering Chengannur (@ceconline.edu)

Project Overview & Presentation Study Guide

How to use this guide: Part 1 explains the project in plain language. Part 2 explains how it is built. Part 3 covers the AI/ML features. Part 4 is the live demo script. Part 5 lists likely viva questions with answers. The appendix has the screenshots to paste into your slides.

1. What is SkillSphere? (in simple words)

SkillSphere is a **website where college students hire each other** for small jobs. Think of it as a mini *Fiverr / Upwork* that works **only inside our college**. A student who is good at poster design, Python, report writing or video editing can list a **service** and set a price. Another student who needs that help can **order it, chat, pay safely, receive the work and leave a review**.

The problem it solves

- Students with skills have no organised way to earn money from them on campus.
- Students needing quick help (a fest poster, a lab report, DSA tutoring) rely on random WhatsApp messages, with no trust, no proof of quality and no payment safety.
- Public freelancing sites are costly, overseas, and not designed for student budgets or campus deadlines.

Our solution

- **Campus-only:** sign-up works only with a college email ending in @ceconline.edu, so everyone is a verified student.
- **Full job lifecycle in one place:** discover, order, chat, deliver, pay, review.
- **Trust:** verified reviews with AI sentiment tags, provider profiles with portfolios, ratings.
- **Smart search:** type what you need in normal English and the system finds relevant services.
- **Safe payments:** Razorpay payment with signature verification and a downloadable receipt.

Who uses it? (three roles)

Role	What they do
Student as Client (buyer)	Searches services, places orders, chats, pays, reviews and can edit their review.
Student as Provider (seller)	Creates services, accepts and delivers orders, replies to reviews, tracks earnings, shows a portfolio. (Any student can be both.)
Admin	Manages users, verifies services, moderates reviews, sees platform statistics.

2. Main features (what you will show)

Feature	What it means	Where in the app
Registration & Login	Only @ceconline.edu emails. Secure login with tokens (JWT). Rate limit stops password guessing.	/register, /login
Service marketplace	Browse, filter by category, price, rating. Each service has description, price, delivery days, revisions.	/services
AI Smart Search	Search in natural language, e.g. 'help with my python assignment'.	/services search bar
Orders	Client places an order with requirements. Status moves: Pending, Accepted, In Progress, Delivered, (Revision), Completed. Cancel or reject possible.	/orders, /orders/:id
Payments & Receipt	Client pays after delivery via Razorpay. A printable receipt shows order no., transaction ID, method, date, amount.	Order page: Pay / Receipt
Reviews with AI sentiment	Client rates 1-5 stars and writes a comment. AI tags it Positive / Neutral / Negative. Client can edit; provider can reply.	Order page, service page
Provider reply	Provider answers a review publicly (and can edit the reply).	Order page (provider login)
Earnings dashboard	Total earned, pending earnings, jobs done, average job value, monthly bar chart, top services.	/earnings
Portfolio & profiles	Student shows past projects with images, tags and links, plus ratings and reviews.	/profile, /users/:id
Chat & Notifications	Direct messages between client and provider; bell icon for alerts.	/messages
Admin dashboard	User control, service verification, flagged review moderation.	/admin

The order journey (story to tell)

1. Client finds a service → 2. places an order → 3. provider **accepts** → 4. provider **starts work** (client can chat) → 5. provider **marks delivered** → 6. client **pays** (order becomes **Completed**, receipt generated, provider's earnings update) → 7. client **reviews** → 8. AI tags sentiment and ratings update → 9. provider **replies**.

3. How it is built (architecture, simplified)

The project follows the common **three-tier** design: a screen the user sees, a brain that applies the rules, and a database that stores data.

Layer	Technology	Simple explanation
Frontend (what you see)	React 18, Vite, Tailwind CSS	React builds the pages as reusable pieces; Vite runs/builds it quickly; Tailwind styles it. Supports dark mode and mobile.
Backend (the brain)	FastAPI (Python), Uvicorn	Receives requests from the website (REST API), checks who you are, applies rules (like 'only client can review'), talks to the database. Auto-generates API docs at /docs.
Database	PostgreSQL + SQLAlchemy (async)	Stores users, services, orders, reviews, payments, chats, portfolios, notifications. 'Async' lets it serve many students at once.
AI / ML	Python (offline)	Sentiment analysis for reviews and TF-IDF semantic search. Runs on our own server, no paid API.
Payments	Razorpay	Creates a payment order; after payment, the server verifies the HMAC signature to prove it is genuine.
Deployment	Docker Compose	Packages frontend, backend and database so it runs anywhere with one command.

Request flow in one line: Browser (React) → HTTP/JSON → FastAPI route → business rules → PostgreSQL → response back to the page.

Database tables (main ones)

Table	Stores
users	name, email, department, role (student/admin), rating, earnings
categories / skills	Design, Writing, Academic, Technology, Media, Electronics and their skills
service_listings	the services providers offer: title, price, delivery days, revisions, rating
orders	who ordered what, price, status, payment status, dates
payment_transactions	Razorpay order/payment IDs, amount, method, status (used for receipt)
reviews	rating, comment, AI sentiment label and score, provider reply
conversations / messages	chat between two users
portfolio_projects	a student's showcase projects (image, link, tags)
notifications	alerts shown under the bell icon

Security measures (good to mention)

- Passwords are stored hashed, never in plain text.
- JWT access token (30 min) + refresh token (7 days), automatic silent refresh.
- Login/registration limited to 10 requests per minute per IP (rate limiting).
- Only @ceconline.edu emails allowed; only the order's client can review; only the reviewer can edit the review; only the reviewee can reply.

- Uploaded images are checked by their real file signature (magic bytes), not just the file name.
- Payment verified using HMAC-SHA256 signature; student profiles require login.

4. The AI / ML features (your highlight)

Key selling point: both AI features run **fully offline on our own server** - zero cost, student data stays private, and responses are instant.

A. Review Sentiment Analysis

What it does: when a student writes a review, the system reads the comment and decides whether it is **positive, neutral or negative**, with a score from -1 (very negative) to +1 (very positive). The service page shows the tag and an overall summary like '94% positive'.

How it works (simple): it uses a **VADER-style lexicon** - a dictionary where each word has an emotion score (e.g. 'excellent' = +, 'terrible' = -). It adds the scores and then adjusts for tricky language: **negation** ('not good' flips meaning), **boosters** ('extremely fast' increases strength) and **CAPITALS** for emphasis. The final compound score decides the label. It also considers the star rating.

Why not ChatGPT/cloud AI? Cost, privacy of student data, speed (<1 ms) and works without internet.

B. Semantic Smart Search

What it does: students can search the way they speak. 'Help with my python assignment' still finds 'Python & ML Assignment Help' even when words differ; 'circuit board' can find 'PCB Design'.

How it works (simple): every service (title + skill + category + description) is turned into a list of numbers using **TF-IDF** - rare, meaningful words get more weight than common ones. The query is turned into numbers too, after adding **synonyms** from 7 campus topics (coding, tutoring, design, writing, video, hardware, maths). Then **cosine similarity** measures how close the query is to each service, and the closest results are ranked first.

Fallback: normal keyword search runs first; if it finds nothing, the semantic engine takes over.

Term	One-line meaning
TF-IDF	A score showing how important a word is in a document compared with all documents.
Cosine similarity	A number from 0 to 1 showing how similar two texts are.
Lexicon	A dictionary of words with pre-assigned sentiment scores.
Compound score	Single final sentiment value from -1 to +1.

5. Live demo script (about 8 minutes)

Accounts (password for all students: **Test@1234**; admin: admin@ceconline.edu / Admin@123).

Step	What to do	What to say
1	Open http://localhost:3000 (landing page).	Introduce the problem and the idea: campus-only skill marketplace.
2	Log in as client **arun.krishnan@ceconline.edu**. Show Dashboard.	Role-based dashboard, stats, recent orders.
3	Go to Services. Type 'help with my python assignment'.	AI semantic search: understands meaning, not just exact words.
4	Open a service detail page, scroll to reviews.	Reviews with AI sentiment tags and the sentiment summary.
5	Open Orders, open a completed order. Show timeline and Receipt.	Full lifecycle; payment verified; printable receipt.
6	On the same order, click Edit review, change text, save.	Reviews are editable; sentiment and ratings recalculate.
7	Log out. Log in as provider **arjun.kumar@ceconline.edu**. Open same order, click Reply to this review.	Provider can respond publicly to feedback.
8	Open Earnings.	Total earned, pending, monthly chart, top services.
9	Open a profile (portfolio).	Students showcase projects to build trust.
10	Log in as admin and show the admin dashboard.	Moderation and platform control.
11	Optional: open http://localhost:8000/docs	Auto-generated API documentation (FastAPI/OpenAPI).

Tip: the order used in these screenshots is arun.krishnan (client) → arjun.kumar (provider). Keep the backend (port 8000) and frontend (port 3000) running.

6. Likely questions and easy answers

Q. Why did you build this?

A. Students have skills but no trusted place to offer them on campus. SkillSphere gives a safe, campus-only marketplace for small paid jobs.

Q. How is it different from Fiverr?

A. Campus-only verified students, student-friendly prices, AI search and sentiment built in, and no external fees.

Q. What technologies did you use and why?

A. React + Tailwind for a fast responsive UI, FastAPI for a quick async Python backend with automatic docs, PostgreSQL for reliable relational data, Razorpay for payments, Docker for easy deployment.

Q. Why FastAPI instead of Django/Flask?

A. It is async (handles many users), has built-in validation with Pydantic and auto API docs, and is easy to pair with Python ML code.

Q. Why PostgreSQL?

A. Our data is relational (users, orders, reviews are linked) and needs transactions and integrity, which PostgreSQL handles well.

Q. How does login work?

A. Email and password are checked; the password is stored hashed. The server returns a JWT access token and a refresh token. The frontend sends the token with every request and refreshes it silently.

Q. How do you ensure only college students join?

A. Registration only accepts emails ending in @ceconline.edu.

Q. Explain the order lifecycle.

A. Pending, Accepted, In Progress, Delivered, optional Revision, then Completed after payment. Cancelled/Rejected are the alternate endings. Each change is checked by the server so only the right person can do it.

Q. How is payment secured?

A. The server creates the payment order with Razorpay, the user pays in Razorpay's checkout, and the server verifies the HMAC-SHA256 signature before marking the order paid. In demo mode a mock checkout is used.

Q. How does sentiment analysis work?

A. A lexicon-based VADER-style model scores words, adjusts for negations, boosters and capitals, and produces a compound score from -1 to +1, which maps to positive/neutral/negative.

Q. How does the smart search work?

A. TF-IDF vectors of services and the query (expanded with synonyms) are compared with cosine similarity and ranked. Keyword search is tried first, semantic is the fallback.

Q. Why not use a big AI model/API?

A. Cost, privacy, speed and offline operation. Our needs are small and well served by lightweight models.

Q. Can a review be edited? Who can reply?

A. Yes, only the client who wrote it can edit it, and ratings recalculate. Only the provider who was reviewed can reply, and can edit that reply.

Q. What security measures are there?

A. Hashed passwords, JWT tokens with expiry, rate limiting, role checks on every action, upload validation by file signature, payment signature verification, login-protected profiles.

Q. How do you handle many users at once?

A. Async FastAPI and async database access let one server handle many concurrent requests without blocking.

Q. What are the limitations and future scope?

A. Payments run in test/mock mode; no real wallet payouts; chat is polling-based. Future: real-time chat (WebSockets), mobile app, escrow payouts, recommendations, dispute resolution.

Q. How did you test it?

A. Automated backend tests with pytest (tests folder) plus manual end-to-end testing with seeded demo data.

Q. How do you run the project?

A. Backend: uvicorn app.main:app --port 8000. Frontend: npm run dev inside the frontend folder (port 3000). Or docker-compose up. PostgreSQL must be running.

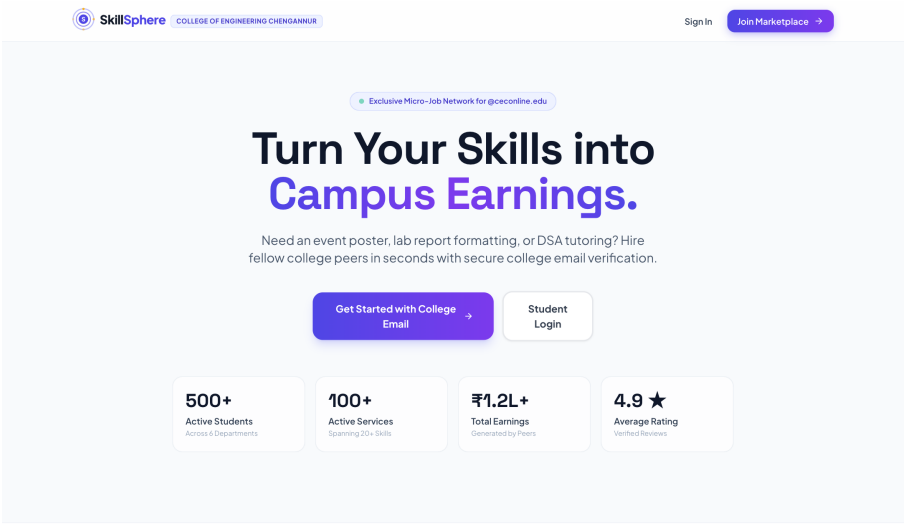
7. One-page cheat sheet

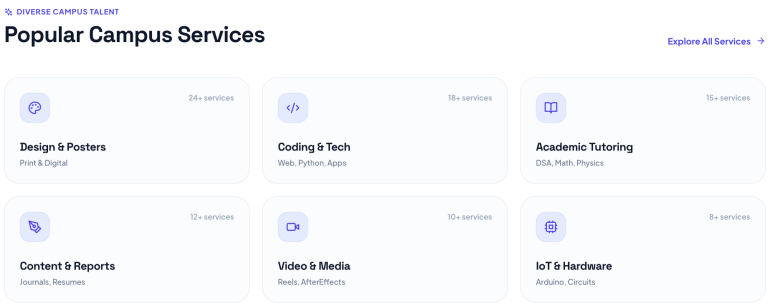
Item	Value
Project	SkillSphere - campus skill exchange marketplace, CEC (@ceconline.edu)
Frontend	React 18, Vite, Tailwind CSS, Axios, React Router, Zustand (auth store), react-hot-toast
Backend	FastAPI, Uvicorn, SQLAlchemy 2 (async), Pydantic v2, JWT
Database	PostgreSQL 15 (asyncpg), Alembic migrations
AI/ML	VADER-style sentiment, TF-IDF + cosine similarity semantic search (offline)
Payments	Razorpay (HMAC verified), receipt generator
Ports	Frontend 3000, Backend 8000, API docs /docs, DB 5432
Logins	Students: *firstname.lastname*@ceconline.edu / Test@1234; Admin: admin@ceconline.edu / Admin@123
Order states	pending, accepted, in_progress, delivered, revision, completed, cancelled, rejected
Review rules	Client only, completed order only, one per order, editable by client, replied by provider

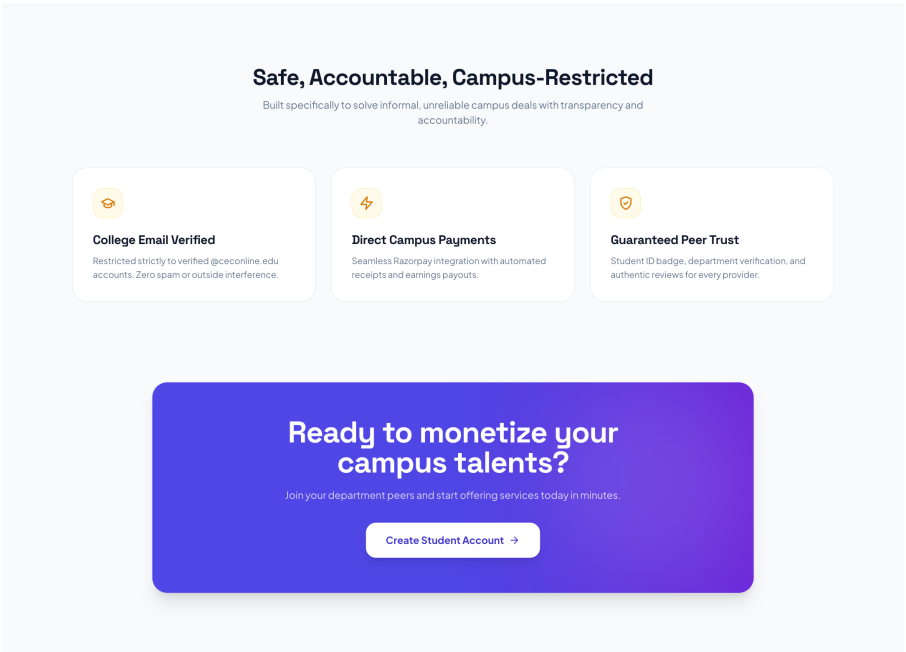
Appendix - Screenshots for your slides

The same images (full resolution) are in the **presentation/screenshots** folder.

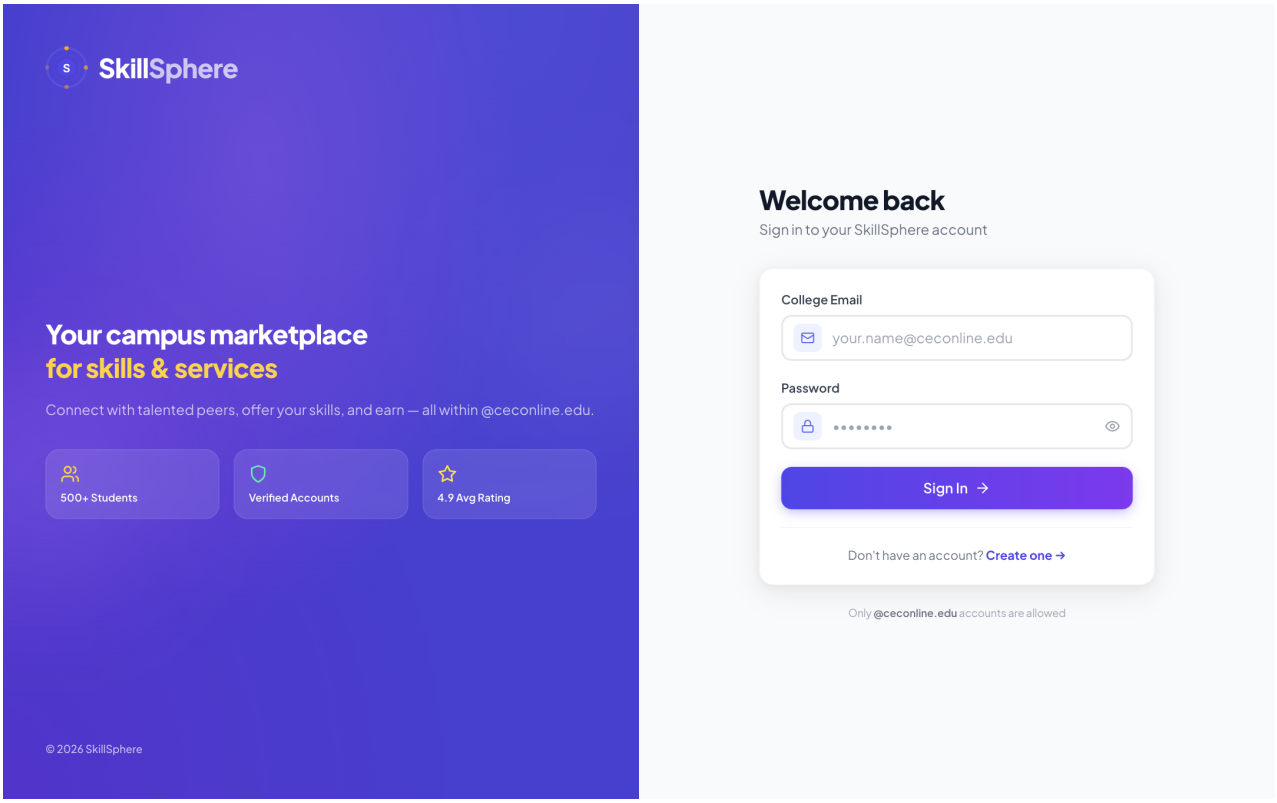
01. Landing page - first impression of the platform.



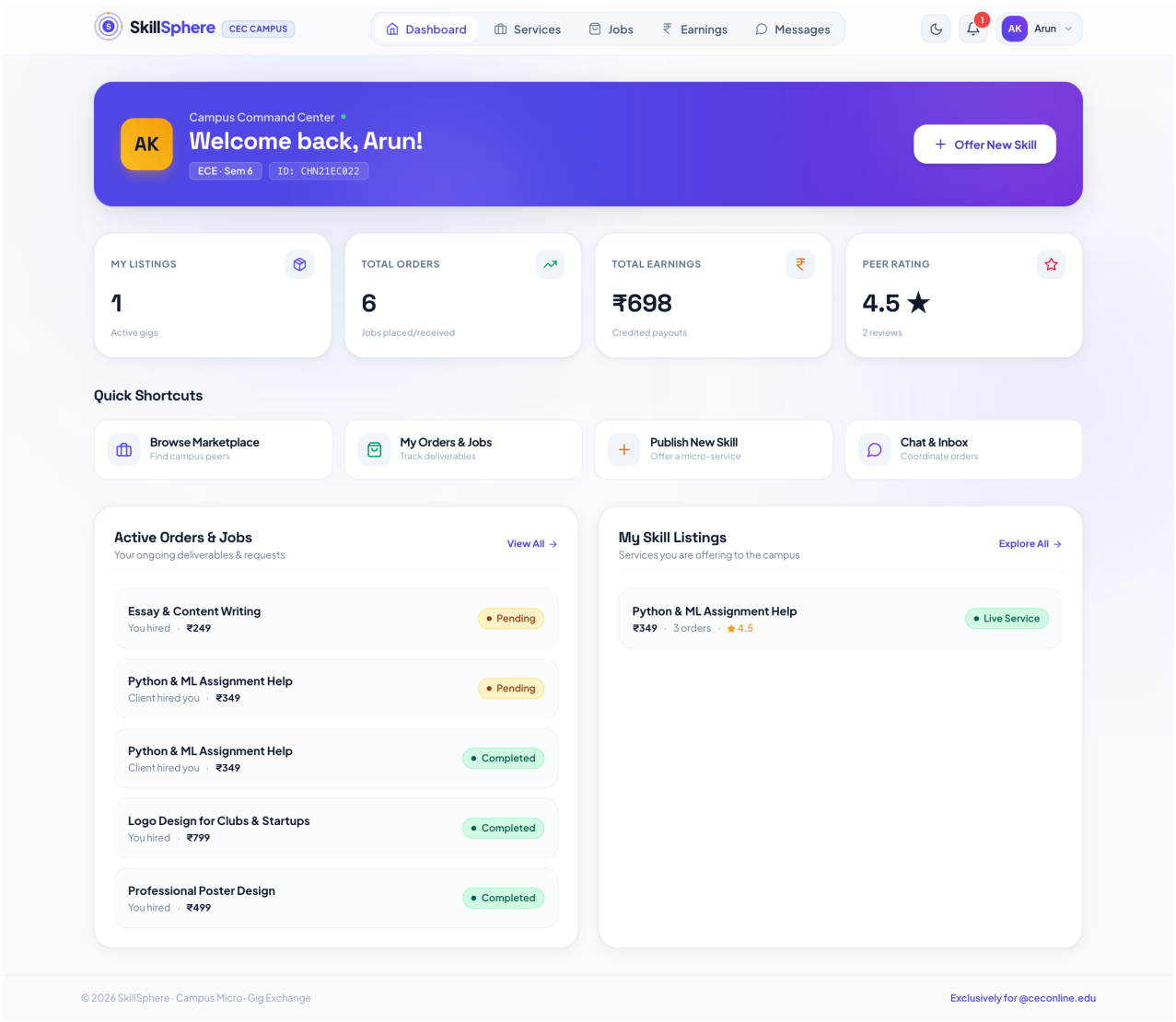




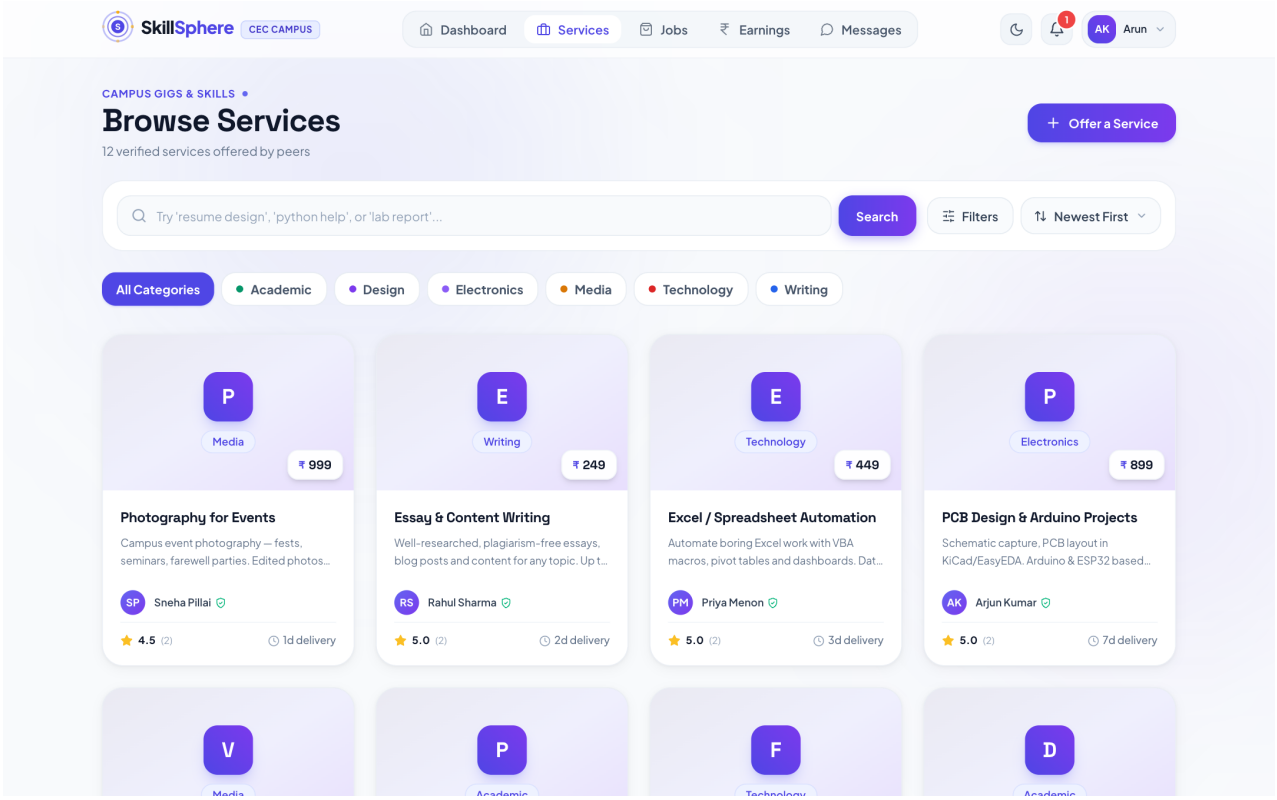
02. Login screen - campus email only.



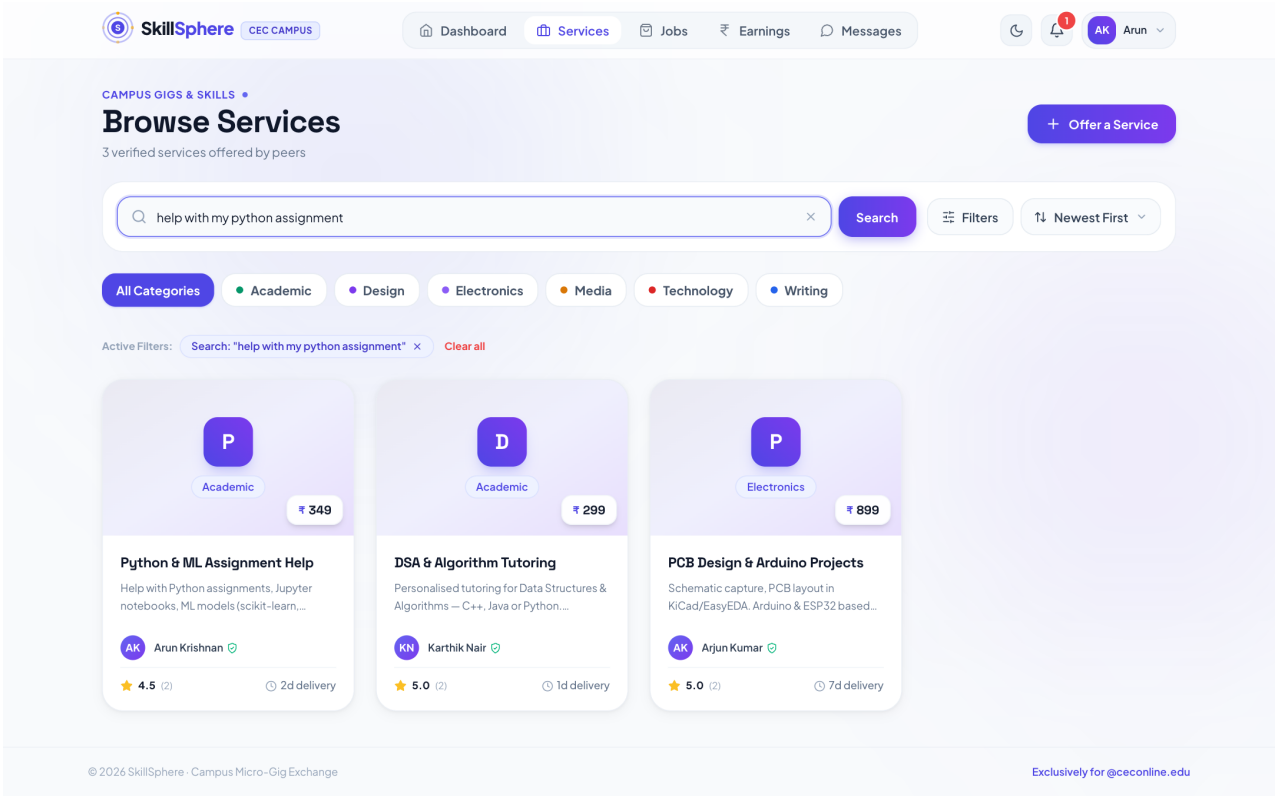
03. Dashboard - overview of orders and activity.



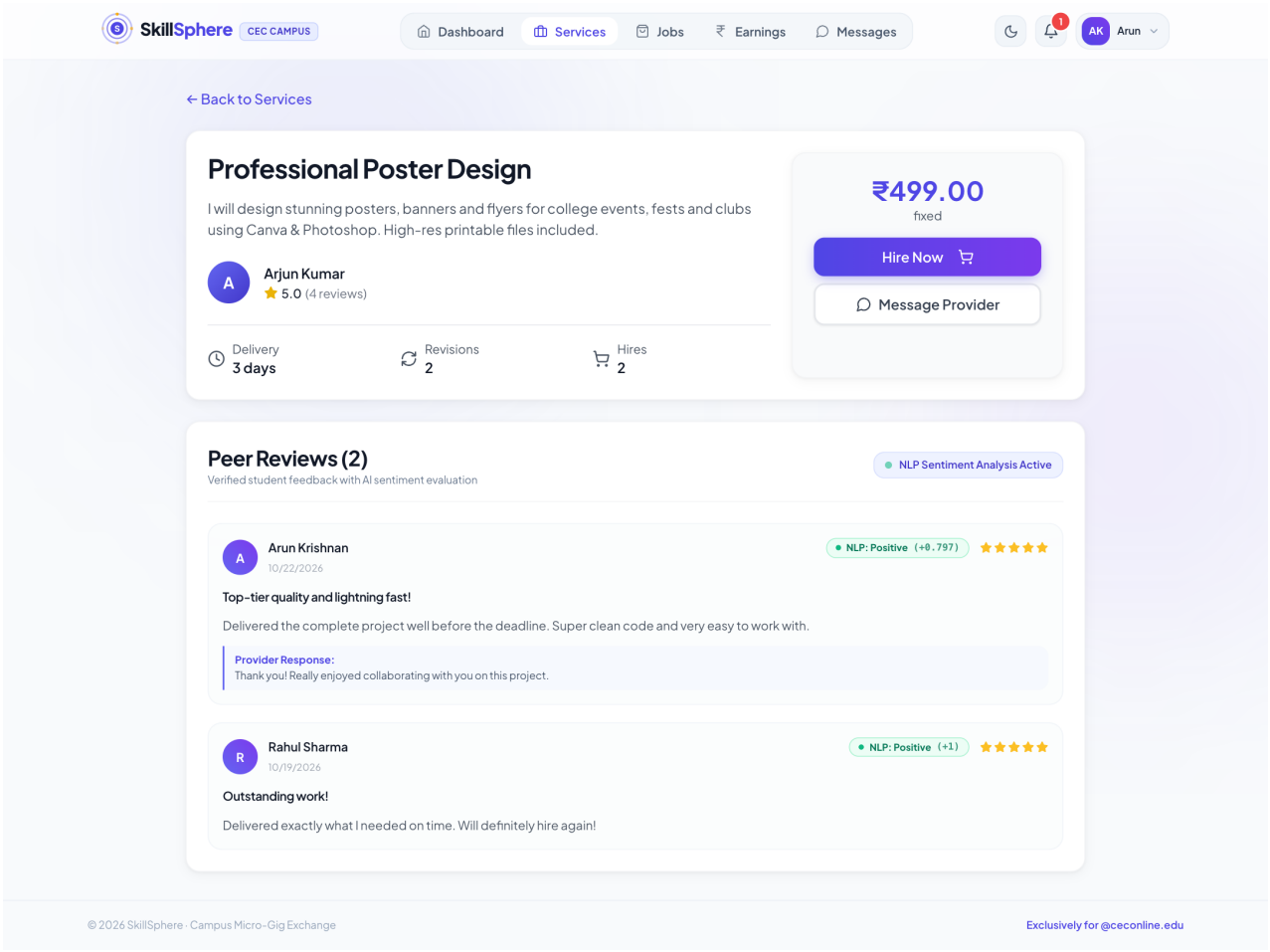
04. Service catalog with filters.



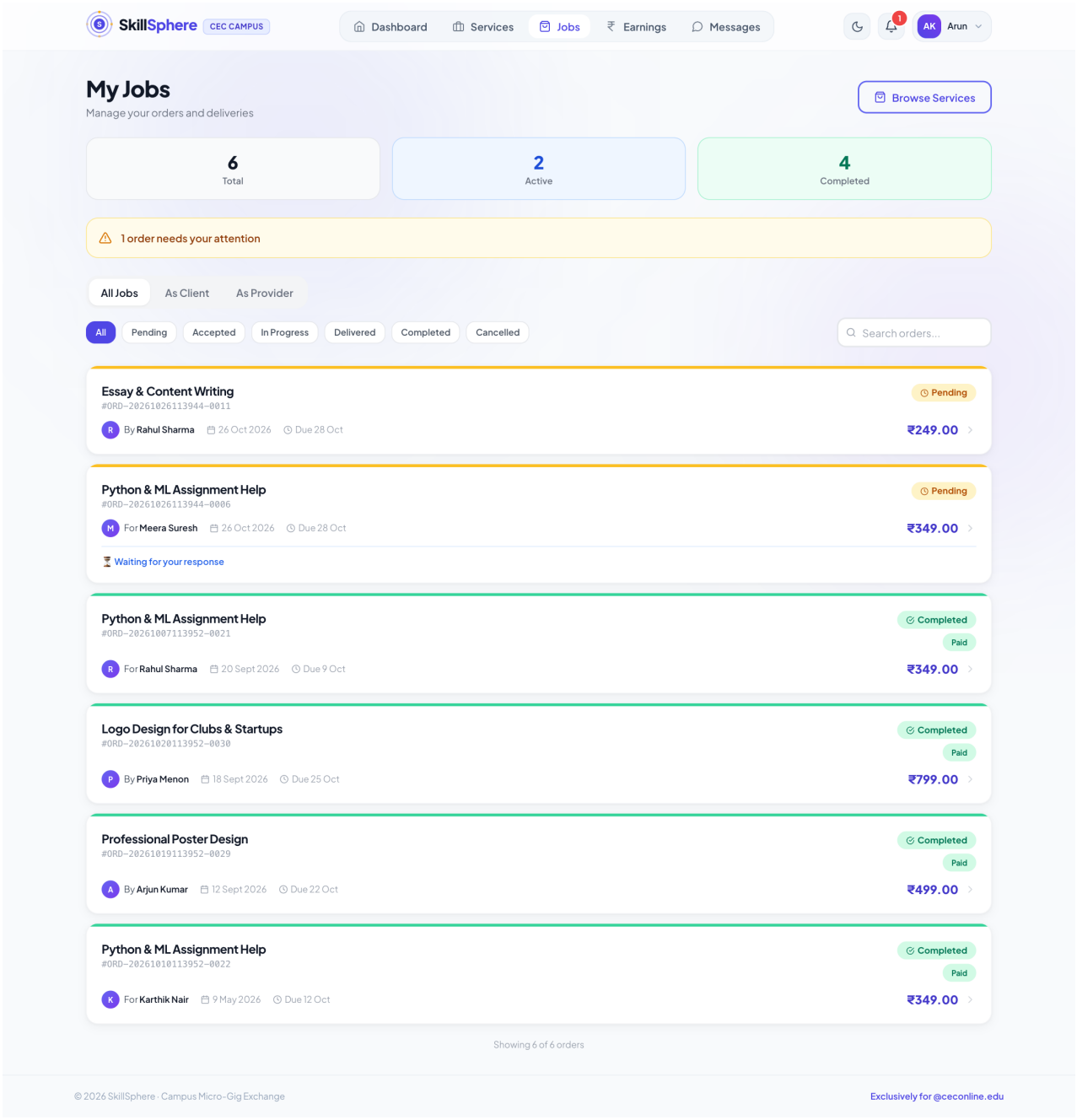
05. AI semantic search - natural-language query.



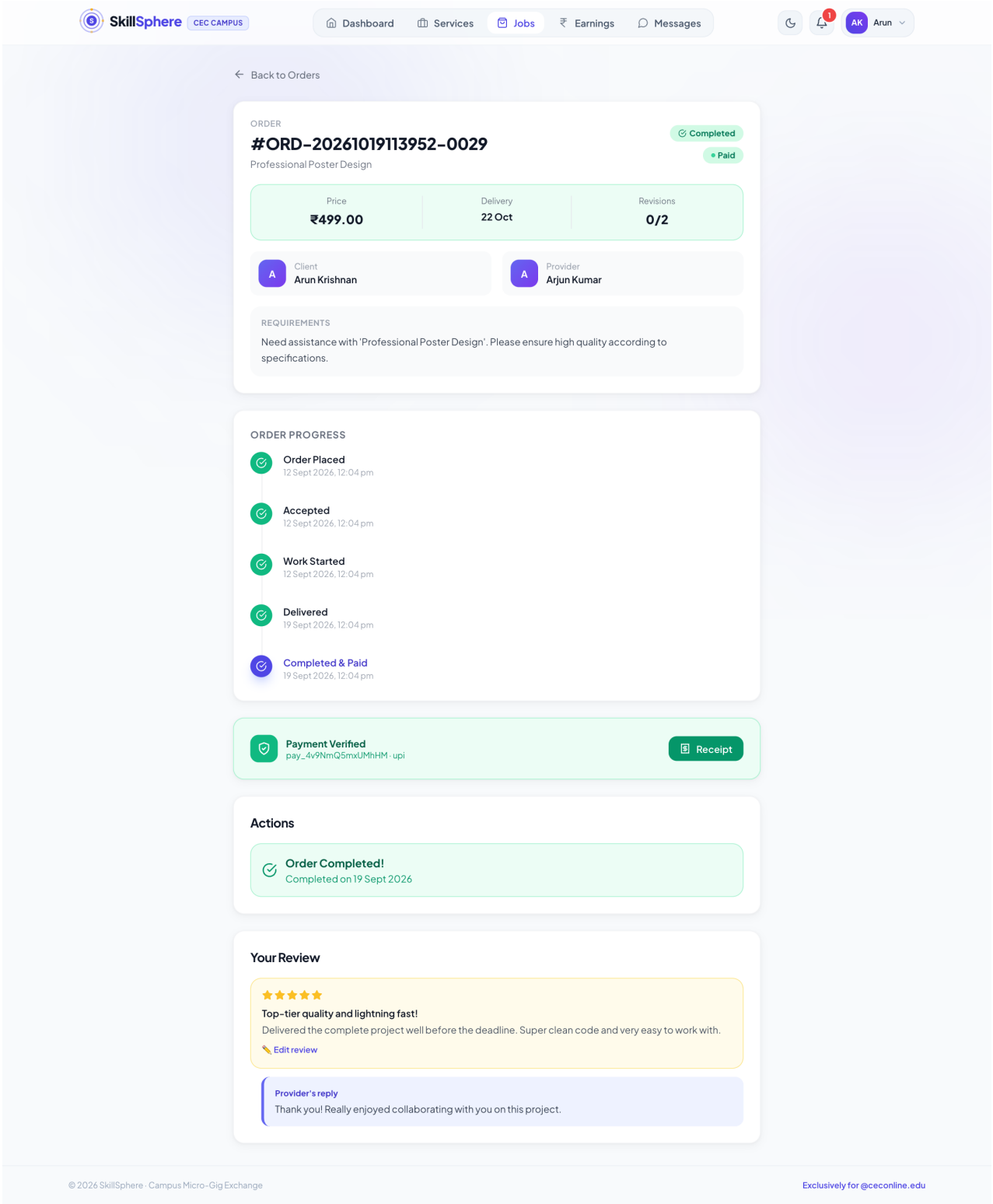
06. Service detail with reviews and AI sentiment.



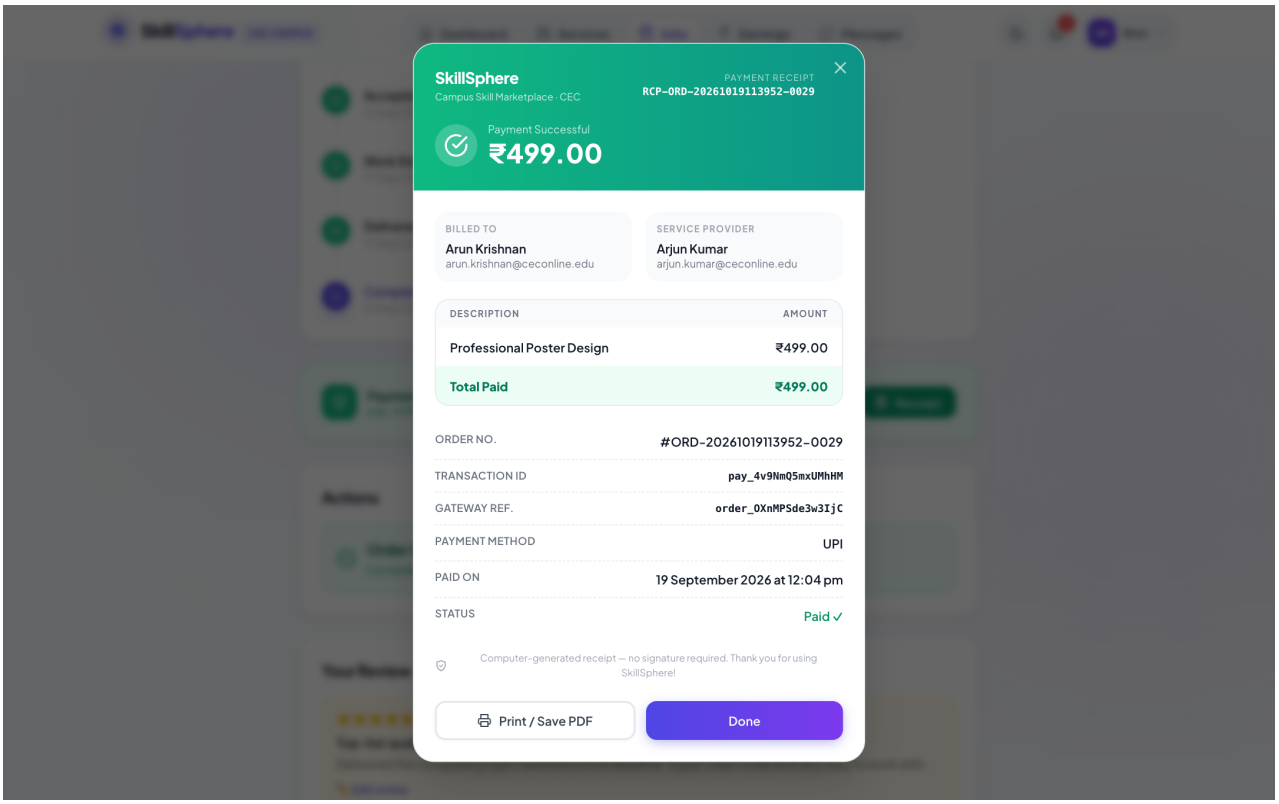
07. Orders list with status badges.



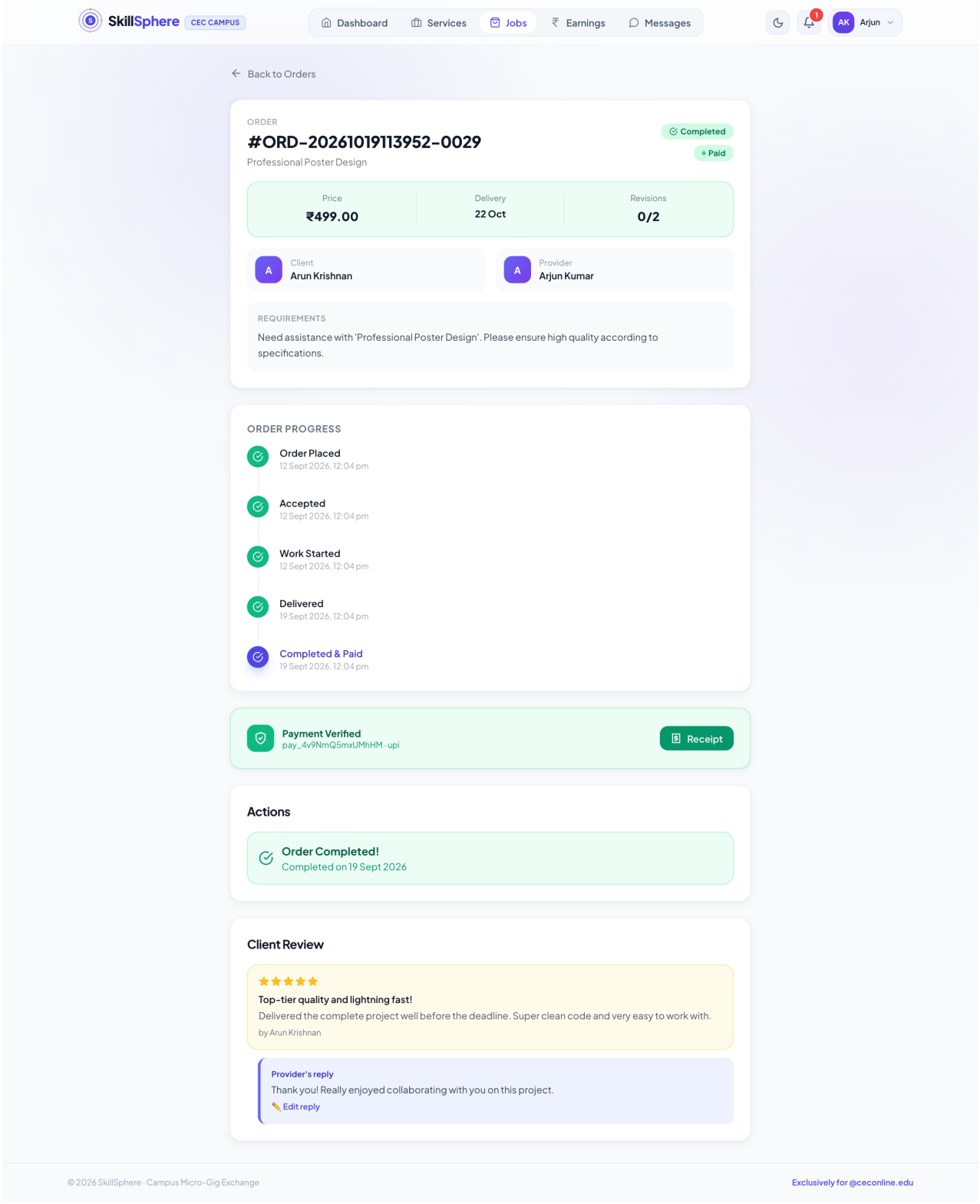
08. Order detail - timeline, payment, review (edit option).



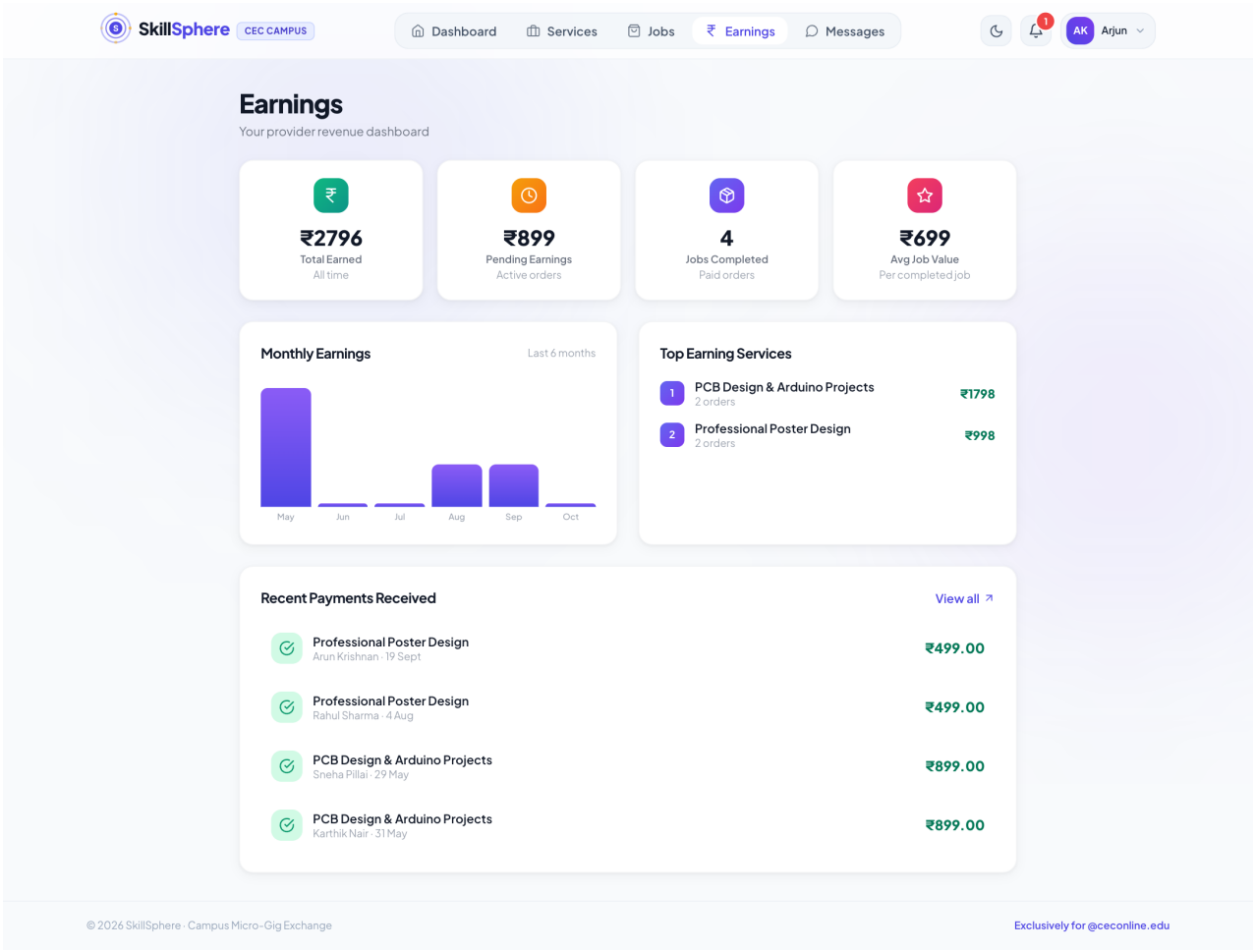
09. Payment receipt (printable / save as PDF).



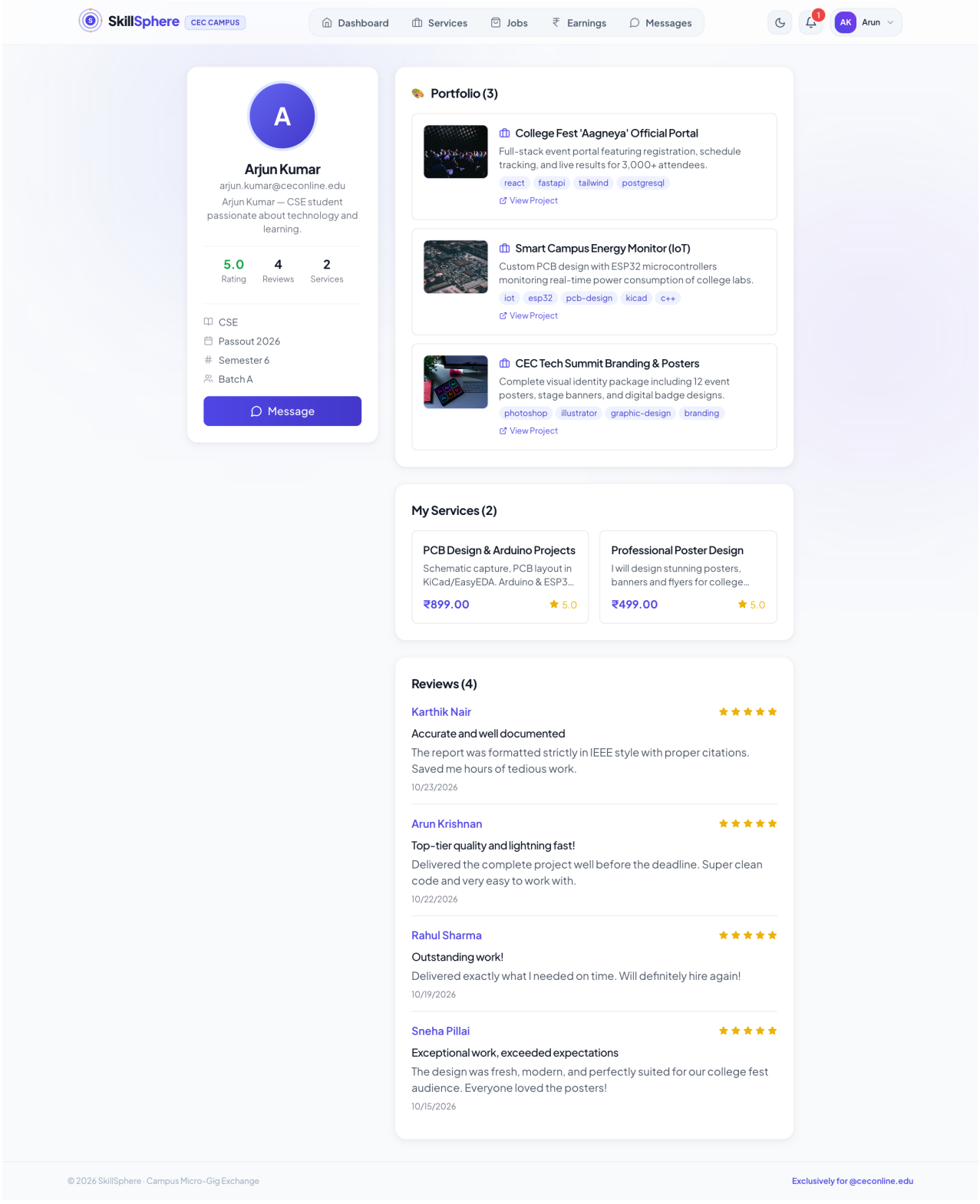
10. Provider view - reply to client's review.



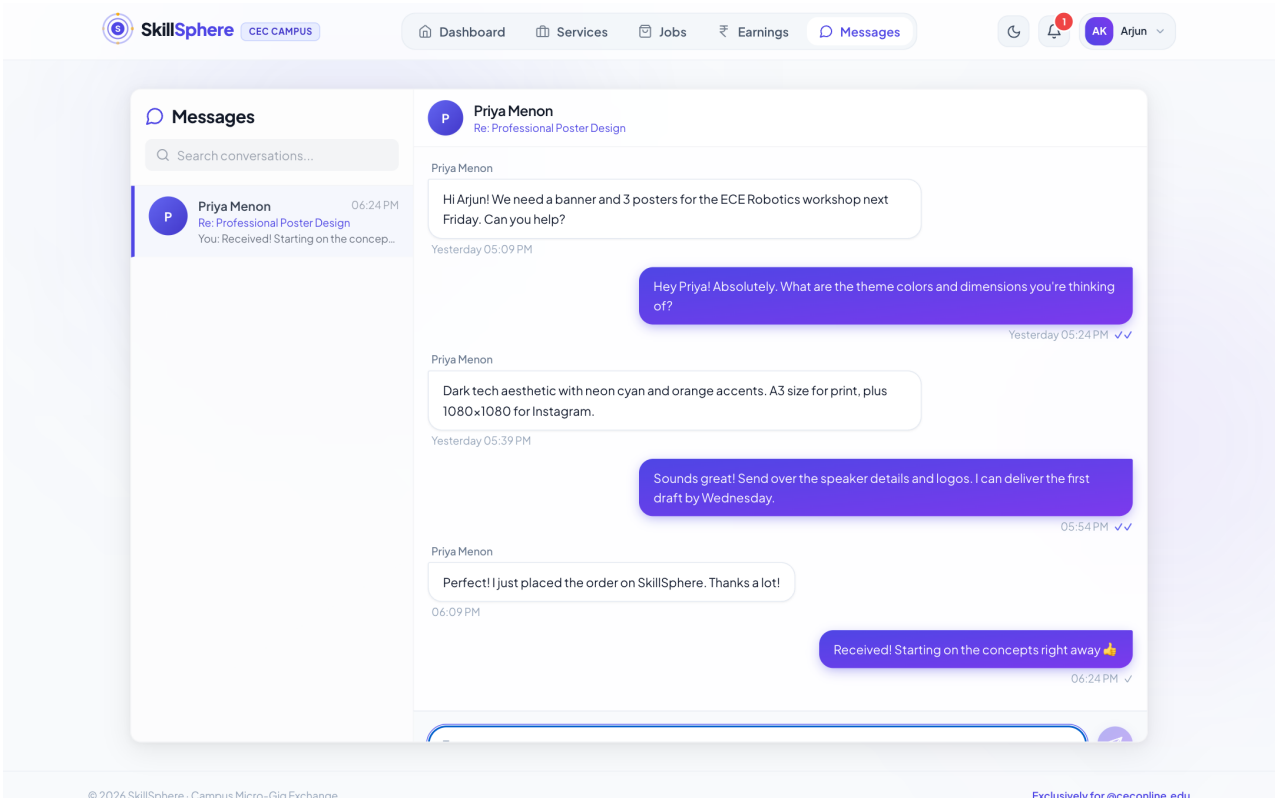
11. Earnings dashboard with monthly chart.



12. Provider profile with portfolio and reviews.

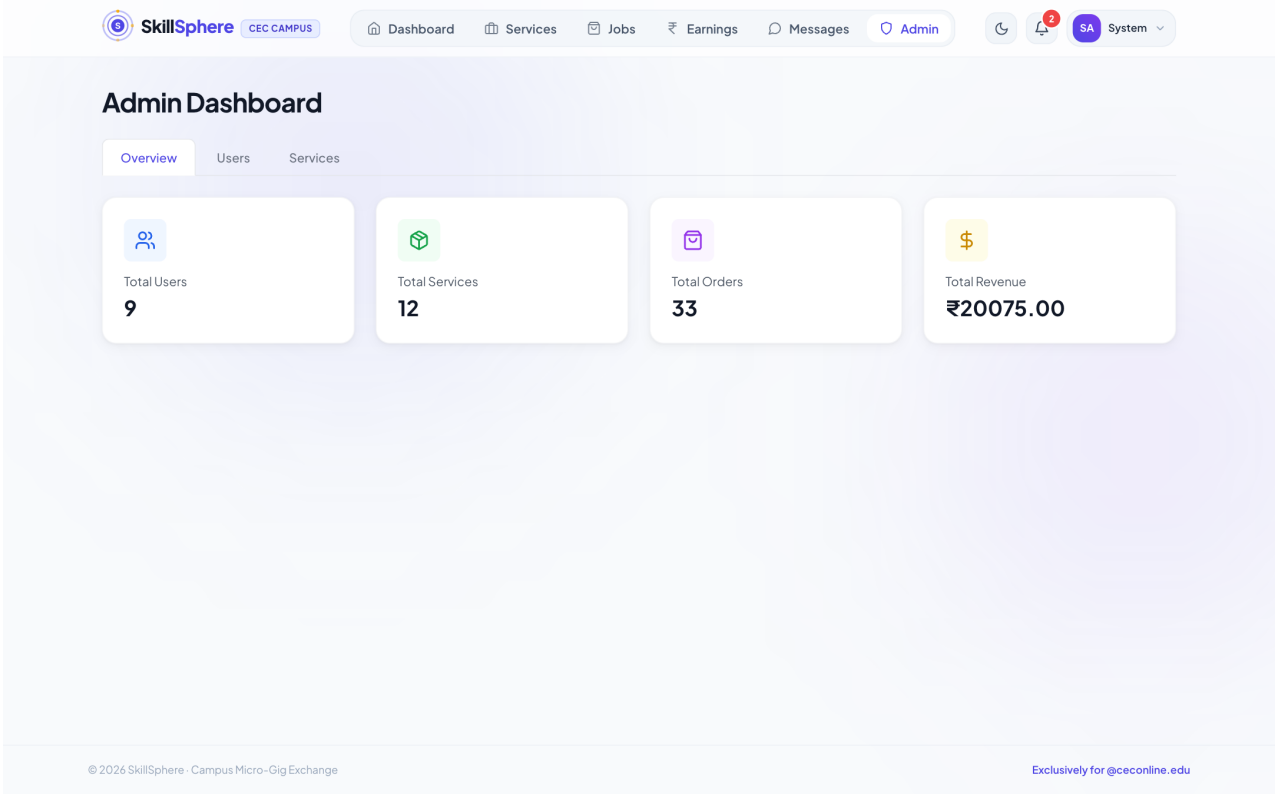


13. Peer chat.



© 2026 SkillSphere · Campus Micro-Gig Exchange
Exclusively for @ceconline.edu

14. Admin dashboard.



© 2026 SkillSphere · Campus Micro-Gig Exchange
Exclusively for @ceconline.edu

15. Auto-generated API documentation (FastAPI /docs).

